Verdigraph Operational Formalization: Mechanically Verified Conservation, Audit, and Safety Invariants in a Self-Evolving AI-Agent Substrate

Justin Hart — *Viridis LLC* — `justin@viridis.security` **Aristotle (Harmonic)** — proof co-author — `aristotle-harmonic@harmonic.fun`

Companion to PAPER_1 (Physical NeuroGenesis), PAPER_2 (Verdigraph Digital NeuroGenesis), and PAPER_3 (Compute Efficiency). Released 2026-05-26.

Abstract

We report the first mechanically verified Lean 4 proofs of three operational invariants of the Verdigraph self-evolving AI-agent substrate: the append-only property of the developmental ledger, the exactness of the publicly-claimed 25% net-revenue conservation routing, and the invariance of the agent's protected node set under pruning. Each invariant is stated as a theorem in Lean 4 + mathlib4, discharged sorry-free, and audited to depend only on the standard mathematical axioms (`propext`, `Classical.choice`, `Quot.sound`). The twelve theorems were produced in sixteen minutes of wall-clock time using Harmonic AI's Aristotle theorem-proving agent. We position the result as a methodological claim more than a mathematical one: formal verification of business-critical AI-agent invariants has crossed the threshold from sponsored research program to routine engineering operation, with per-proof cost measured in minutes rather than months. We describe the audit protocol that turned the agent's output into a deposit-grade artifact, the spec-invariance work that remains irreducibly human, and the implications for any AI-agent framework that wishes to ground public trust claims in mechanized proof rather than in marketing language.

1. Introduction

The companion papers in this series describe Verdigraph as a developmental software framework in which an AI agent grows its own cognitive substrate (PAPER_2), and an efficiency layer that routes each task along the cheapest reliable cognitive pathway under a hard minimum-quality contract (PAPER_3). Three properties of this framework are load-bearing for the trust story it tells: that the developmental ledger is genuinely append-only, that exactly twenty-five percent of net revenue is routed to conservation programs, and that the agent's safety-axiomatic protected nodes are never silently pruned.

These properties have, until now, been informal claims. They appear in documentation, in the project's landing page, in launch material, and in software invariant comments. Each is checked by unit tests of the corresponding Python implementations. None had been stated as a theorem and discharged by a proof assistant.

This paper closes that gap for the three properties named above. We define each property in Lean 4 over a model of the relevant agent component, state the theorem, present the proof discharged by the Aristotle theorem-proving agent, and audit the result. The accompanying Lean source is published as part of the Verdigraph code base under MIT license; the Aristotle run identifiers and per-run postmortems are included as supplementary material so that any reader can reproduce, inspect, or extend the formalization.

The substantive scientific claim of this paper is narrow: the three theorems hold in the stated model. The substantive methodological claim is broader: the cost of formally verifying invariants of this kind, when phrased in mathematical primitives that Lean's mathlib already covers, is now measured in tens of minutes per proof, not the months or years that prior

formal-methods literature has reported. We argue that this changes what is reasonable to expect of any AI-agent framework that makes public claims of trust, safety, or accounting precision.

2. Core Thesis

> Every publicly-stated trust claim of an AI-agent framework that can be expressed as a mathematical invariant should be mechanically verified before it is asserted to the public, and the proof should be deposited alongside the software as a citable, independently verifiable artifact.

The thesis is normative, not merely descriptive. We do not claim that every agent framework currently does this; we claim that the cost calculus has shifted to the point where not doing it is no longer defensible for load-bearing claims. The remainder of the paper substantiates the cost claim by demonstrating it on three Verdigraph invariants, and discusses the implications for the broader AI-agent ecosystem.

3. Background: The Intelligence Bound Corpus

Verdigraph's verified layer does not begin with the work reported here. It extends a prior corpus of Lean formalizations of the Intelligence Bound, the thermodynamic-economic bound on useful information output of a dissipative computational system. The bound, derived in the supporting physics work and stated informally as

> dI/dt ≤ P · D / (k_B T · ln 2)

upper-bounds the rate of useful information production by a system with dissipation power P, dissipation factor D, and thermal energy $k_B T$. The bound is axiomatized in Lean as `landauer_bound` and serves as the single physical axiom for the downstream corpus.

The Intelligence Bound corpus, formalized as projects T-IB-01 through T-IB-09 over a period of weeks in May 2026 and verified sorry-free with Aristotle assistance, contains the following theorem families:

Project	Substance
T-IB-01	Attribution conservation: captured-bit budget ≤ Intelligence Bound budget − authorized-bit budget. The actuarial primitive for cyber-insurance underwriting.
T-IB-02..08	Adversarial dissipation, asymmetric defense, low-power capture-surface bounds, and related corollaries.
T-IB-09	Attack-irrationality threshold: above an attacker-multiplier M, adversarial campaigns become economically irrational under bounded budgets.
PSIT_Symplectic	Phase-Space Intelligence Theory: symplectic-geometric foundation including a linear Darboux theorem and Pfaffian-based uniqueness for 4-dimensional skew-symmetric forms.

The present paper introduces the **T-VG** (Verdigraph) tier, which sits operationally above the IB corpus: it formalizes properties of the running software substrate that *embodies* an IB-compliant agent. T-VG-01 establishes the trust substrate (the ledger). T-VG-02 instantiates

the T-IB-01 attribution-conservation pattern in the revenue-allocation domain (the twenty-five percent split). T-VG-03 formalizes a graph-level safety invariant (protected nodes survive pruning) that mirrors, in a different domain, the adversarial-resistance theme of T-IB-09.

4. The Three Operational Theorems

We summarize each theorem and its proof in this section. Full Lean source is included in the accompanying repository under `lean/verdigraph/`.

4.1 T-VG-01: Ledger Append-Only

Informal statement. Any operation applied to a developmental ledger either leaves the ledger unchanged or appends exactly one event to its end. The original event sequence is a prefix of the resulting event sequence; no event is ever modified, removed, or reordered.

Lean formulation. The ledger is modeled as a list of timestamped events with a carried monotonicity invariant. The set of allowed operations is `{ noop, append(e) }`. The `apply` function rejects appends whose timestamp precedes the ledger's last timestamp (returning the ledger unchanged) and accepts those that respect monotonicity (appending the event).

The principal obligation is the `apply_preserves_prefix` theorem:

```
theorem apply_preserves_prefix (L : DevelopmentalLedger) (op : LedgerOp) :
    L.events <+: (apply L op).events
```

with corollaries `apply_never_shortens` (length monotonicity) and `append_succ_or_noop` (appends either add exactly one event or leave the ledger unchanged).

Result. All three obligations and the embedded `monotone_ts` proof inside `apply` are discharged sorry-free. The non-trivial proof in the appended-event branch case-splits on whether each comparison index falls within the original event list or points to the newly appended event, using the carried `monotone_ts` invariant for the original-events case and transitivity through `lastTimestamp` for the boundary case.

4.2 T-VG-02: Conservation Share Exactness

Informal statement. For every completed-payment revenue event, twenty-five percent of net revenue is routed to conservation. No money is created or destroyed: the conservation share plus the retained share sums to net revenue exactly. The conservation share equals the integer floor of one quarter of net revenue, with the floor residual (at most three cents per event) remaining in the retained bucket.

Lean formulation. A revenue event carries gross, Stripe-fee, and refund amounts in integer cents (USD), with the obvious non-negativity and ordering invariants. Net revenue is defined as

```
def netRevenue (e : RevenueEvent) : Nat :=
  e.gross_cents - e.stripe_fee_cents - e.refund_cents
```

and the conservation share is defined as

```
def conservationShare (e : RevenueEvent) : Nat :=
  netRevenue e / 4
```

using Nat integer division (i.e., floor). The five obligations are `share_partition`, `share_lower_bound`, `share_upper_bound`, `share_exactness` (the conjunction), and `share_monotone` (revenue monotonicity of the share).

Result. All five theorems are discharged sorry-free using standard mathlib lemmas on natural-number division (`Nat.add_sub_cancel'`, `Nat.div_le_self`, `Nat.div_mul_le_self`, `Nat.div_add_mod`, `Nat.mod_lt`, `Nat.div_le_div_right`). The headline `share_partition` theorem reduces to a single application of `Nat.add_sub_cancel'`:

```
theorem share_partition (e : RevenueEvent) :
    conservationShare e + retained e = netRevenue e :=
  Nat.add_sub_cancel' (Nat.div_le_self _ 4)
```

This one line is the formal proof that the publicly-claimed conservation math conserves every cent.

4.3 T-VG-03: Pruning Protected-Set Invariance

Informal statement. For any cognitive graph and any sequence of pruning operations governed by the agent's growth rules, the set of protected nodes declared in the agent's safety axioms is preserved.

Lean formulation. Rather than fix a specific pruning implementation in the theorem statement, we factor the property into a predicate `SpecConforming` that any pruning step must satisfy:

```
def SpecConforming (step : CognitiveGraph → CognitiveGraph)
    («protected» : Finset NodeId) : Prop :=
  ∀ G : CognitiveGraph,
    («protected» ⊆ G.nodes → «protected» ⊆ (step G).nodes) ∧
    (step G).nodes ⊆ G.nodes ∧
    (step G).edges ⊆ G.edges
```

This is intentional: it decouples the theorem from the specific implementation in `verdigraph/pruning.py`, so the theorems remain applicable as long as the implementation refines the predicate. The four obligations are `protected_invariance` (single-step), `protected_invariance_iter` (any number of iterations), `nodes_monotone`, and `edges_monotone`.

Result. All four are discharged sorry-free. Three are single-line projections out of the `SpecConforming` conjunction; `protected_invariance_iter` is a one-line induction. The mirror-image structure of the proofs reflects the deliberate engineering choice to bake the obligations into the predicate rather than into the theorem hypotheses.

5. Methodology

The three projects were scaffolded by hand against the existing T-IB-01 project layout (Lean 4.28.0, mathlib4 pinned to the same revision). For each, we wrote the structures, definitions, and theorem statements, leaving every proof obligation as `sorry`. Each project also contained a `README.md`, an `ARISTOTLE_NOTES.md` documenting the spec defaults we had adopted, and a header comment in the `.lean` file naming the corpus parent. The three projects were submitted in parallel to Aristotle via the `aristotlelib` Python SDK using `Project.create_from_directory`, with prompts that named the theorem obligations explicitly and forbade any weakening of the signatures.

The three runs returned with all twelve obligations discharged sorry-free. Wall-clock times from submission to artifact availability were: T-VG-02 (eight minutes, five theorems, all single-tactic mathlib closures), T-VG-01 (fourteen minutes, four obligations including the non-trivial `monotone_ts` discharge), and T-VG-03 (sixteen minutes, four obligations plus a syntactic keyword-escape fix). Total wall-clock for the batch was sixteen minutes.

Each returned `.lean` file was subjected to a structural audit before acceptance into the repository. The audit consisted of: (a) confirming zero `sorry` occurrences in the returned source;

(b) confirming zero new `axiom` declarations; (c) confirming zero uses of `admit`, `unsafe`, or `native_decide` (the three principal tactics that bypass mathematical content); (d) a line-by-line diff of every top-level declaration's signature against the scaffold to confirm that no theorem had been weakened; and (e) review of each proof body for plausibility and mathlib-idiomatic structure.

One semantic-preserving change was disclosed by Aristotle and verified during audit: the identifier `protected` was renamed to «`protected`» (the Lean 4 keyword-escape form) throughout T-VG-03, because `protected` is a reserved keyword in Lean 4. This was a syntactic correction of a scaffold bug, not a theorem modification.

6. Spec Invariance: The Irreducibly Human Work

The proofs are correct for the theorems they state. They are not self-evidently correct for the public-facing claims they are intended to underwrite. The gap between the two is the spec-invariance problem: did we, in stating the Lean theorem, capture the same property that the landing-page copy, the registry submission, and the customer contract language describe?

We do not solve this problem in this paper. We document it explicitly so that future readers, and future revisions of this corpus, treat it as the primary risk surface. For each theorem we record the spec defaults we adopted in the corresponding `ARISTOTLE_NOTES.md`. Of these, the highest- stakes for T-VG-02 are:

- **Q2.1 Net revenue definition.** We adopted `net = gross − stripe_fee − refunds`, the standard SaaS net-revenue accounting convention. The alternative — Viridis absorbs the Stripe fee, making `net = gross − refunds` — would require modifying the `RevenueEvent` structure and re-running the proof (an additional eight minutes of Aristotle time).
- **Q2.4 "Twenty-five percent of NET" as the public claim.** We adopted the wording in the project's operator skill specification, which says "twenty-five percent of net completed-payment revenue." Drift between marketing copy and the Lean spec is the failure mode that this verification layer is least able to detect, and that periodic copy-review is required to detect.

A general procedure for any future verified claim of this kind is suggested: (i) freeze the spec in `ARISTOTLE_NOTES.md` before submitting to Aristotle; (ii) audit returned proofs structurally and against the spec; (iii) periodically walk the public-facing copy against the spec, with the burden of proof on the claim author to demonstrate alignment.

7. Operational Implications

We identify five operational implications of the result.

First, mechanized claims become contractual terms. A customer or counterparty can stipulate that a claim be supported by a mechanized proof, and that the proof be deposited alongside the deliverable. The total cost of meeting this requirement is now bounded by the time to freeze the spec, scaffold the Lean project, and submit to a theorem-proving agent — a multi-day, not multi-month, undertaking for invariants of the present complexity class.

Second, drift becomes a build break. Continuous-integration tooling can be configured to rebuild the Lean proofs on every change to a load-bearing file (e.g., `verdigraph/ledger.py`, `verdigraph/billing/conservation.py`, `verdigraph/pruning.py`). A change that violates the spec breaks the build. The verified layer thus becomes a guardrail against silent regression.

Third, regulator and procurement-grade documentation becomes affordable. Formal-methods language has historically been the province of safety-critical aerospace, medical-device, and defense procurement contexts where the compliance cost is amortized over the program lifetime. The cost per verified invariant is now low enough that formal-methods

language becomes appropriate for routine commercial procurement involving AI-agent infrastructure.

Fourth, audit and underwriting acquire a citable substrate. Each deposited proof carries a DOI (via Zenodo) and an Aristotle run identifier; both are independently verifiable. Cyber-insurance underwriters, ESG-fund custodians, and procurement auditors can reference the deposit identifiers in their own documentation. The corresponding T-IB-01 actuarial primitive becomes operationally citable in policy documents.

Fifth, the verification practice compounds. Each invariant proved becomes a reusable structural primitive for future proofs. The `SpecConforming` pattern introduced in T-VG-03 can be reused for any spec-vs-implementation invariance theorem. The conservation-partition pattern in T-VG-02 can be reused for any fixed-ratio routing invariant. A library of such patterns lowers the marginal cost of each subsequent proof.

8. Connection to the IB Corpus

T-VG-02 is the operational descendant of T-IB-01. The shape of the two theorems is the same: a partition of a quantity into authorized and allocated components, with an invariant relating their sum to a budget. T-IB-01 bounds the captured-bit share by the Intelligence Bound budget minus the authorized-bit share. T-VG-02 fixes the conservation share at exactly one quarter of net revenue. The difference between an upper-bound invariant and an exactness invariant is captured cleanly in Lean by the choice of $\leq$ versus $=$; the structural similarity allows the two theorems to share notation, comment headers, and audit protocol.

T-VG-01 has no direct parent in the IB corpus. It is the substrate on which any audit claim in any T-VG theorem rests: a conservation theorem is worthless if the ledger that records the conservation events can be tampered with. We classify it as Verdigraph-native and treat it as the ground floor of the operational tier.

T-VG-03 has no direct parent either, but it stands in structural analogy to T-IB-09. T-IB-09 bounds an adversary's economic spend under adversarial- dissipation hypotheses; T-VG-03 bounds graph-level mutation reachability under safety-axiom hypotheses. Both theorems express a "cannot harm what is protected" invariant in their respective domains, and we expect future theorems to deepen this analogy.

The combined corpus thus has a recognizable shape: physical bound (Phys Rev E paper, axiomatized as `landauer_bound`) → mathematical theorems about the bound (T-IB-01..09) → geometric foundation (PSIT_Symplectic) → operational embodiment in software (T-VG-01..03). The argument from physics to verified production code is, for the first time, fully closed.

9. Relation to Existing Work in Formal Methods

The formal methods community has, over decades, developed mechanized verification practices for compilers (CompCert), microkernels (seL4), distributed systems (Verdi, IronFleet), and cryptographic protocols (F*, EasyCrypt). The present work does not advance the state of the art in any of these directions. The contributions here are methodological and applied, not foundational.

The novel element is the application context. To our knowledge, no prior published AI-agent framework includes mechanized proofs of its public trust claims. The closest analogues are formally verified ML runtimes and formally verified differential-privacy mechanisms, but neither extends to the agent-orchestration layer at which Verdigraph operates.

The Aristotle theorem-proving agent has been described in recent literature, including a public report of gold-medal-equivalent performance on the 2025 International Mathemati-

cal Olympiad problems. We treat Aristotle here as a tool: our claims are about what becomes *operationally possible* with such a tool, not about the tool's inner workings.

10. Reproducibility

All sources for the three theorems are deposited under the repository's `lean/verdigraph/` directory and under the corresponding Zenodo deposit manifest. To reproduce:

```
git clone https://github.com/viridis-security/verdigraph-neurogenesis
cd verdigraph-neurogenesis/lean/verdigraph/T-VG-01-ledger-append-only
lake build
# repeat for T-VG-02 and T-VG-03
```

The `SUBMISSIONS.json` file records the Aristotle project identifiers, run identifiers, completion timestamps, and audit summaries for each of the three runs. The `poll.py` script in the same directory can be used to re-fetch the original Aristotle artifacts from the API given a valid Harmonic API key.

To re-run the proofs (as opposed to merely re-checking them), the scaffolds are also preserved with their `sorry` placeholders intact in the git history; the commit immediately preceding `5566242` contains the unfinished scaffolds.

11. Future Work

We identify three lines of work that follow naturally from this paper.

Implementation-to-spec linkage. The Lean proofs hold for the model defined in the corresponding `.lean` files. They do not establish that the production Python implementations in `verdigraph/ledger.py`, `verdigraph/billing/conservation.py`, and `verdigraph/pruning.py` faithfully refine the model. Property-based testing keyed to the Lean specs is the next operational layer; eventual extraction of executable specifications from the Lean source is the longer-term goal.

Verified persistence. The operator agent's daily-run state file is currently committed to git with no enforced persistence semantics. An architectural decision record proposing externalization to Cloudflare D1 with Lean-derived CHECK-constraint and trigger generation is in preparation; the goal is to have the database layer enforce the properties that Lean has proved.

Extension of the T-VG tier. Natural next theorems include routing-safety properties (related to the compute-efficiency layer described in PAPER_3), refund-handling correctness (closing T-VG-02 against the full Stripe event lifecycle), and conservation under multi-currency operation (relaxing the USD-only assumption of the present T-VG-02). Each is in scope for the present verification practice and estimated to require under an hour of Aristotle time per theorem family.

12. Conclusion

We have presented mechanized Lean 4 proofs of three operational invariants of the Verdigraph AI-agent substrate: ledger append-only, twenty-five- percent conservation share exactness, and pruning protected-set invariance. Each invariant was discharged sorry-free, audited to depend only on standard mathematical axioms, and integrated into the project's source tree as a citable artifact. The wall-clock time from submission to deposit-ready state, for all twelve theorems combined, was sixteen minutes.

The narrow scientific claim of this paper is that the three named invariants hold in the stated model. The broader methodological claim is that the practice of mechanically verifying AI-

agent invariants of this class has crossed a cost threshold below which it becomes routine engineering rather than sponsored research. We argue that any AI-agent framework that makes public claims of trust, safety, or accounting precision should now ground those claims in mechanized proof rather than in marketing language, and we present this corpus as one model of how such grounding can be accomplished.

References

1. Hart, J. *Viridis NeuroGenesis: A Physical Concept for AI-Designed Brains Grown in Solution and 3D-Printed Scaffolds.* PAPER_1 of this corpus. 2026.
2. Hart, J. *Verdigraph: A Software Framework for AI-Agent-Architected, Self-Evolving Cognitive Substrates.* PAPER_2 of this corpus. 2026.
3. Hart, J. *Verdigraph Compute Efficiency: A Self-Optimizing Cognitive Routing Layer for Local GPU and Transformer-Based AI Agents.* PAPER_3 of this corpus. 2026.
4. Hart, J. *The Intelligence Bound: A Thermodynamic Upper Bound on Useful-Information Production in Dissipative Computational Systems.* Physical Review E (under review). 2026.
5. Hart, J. *Compute is Carbon: Why AI Needs a Thermodynamic Floor.* Nature Sustainability (forthcoming). 2026.
6. The mathlib Community. *The Lean Mathematical Library.* Various dates; pinned at revision matching `lakefile.toml` in this deposit.
7. Harmonic AI. *Aristotle: IMO-Level Automated Theorem Proving.* arXiv preprint 2510.01346. 2025.
8. Verdigraph Source Repository. `https://github.com/viridis-security/verdigraph-neurogenesis`. T-VG proofs at `lean/verdigraph/`.

Submission tracking, Aristotle run identifiers, per-run postmortems, and audit metadata are included in the deposit at `lean/verdigraph/SUBMISSIONS.json` and in each project's `ARISTOTLE_SUMMARY.md`. The deposit version is 0.2.0, keyed to the v0.1.0 deposit (DOI to be reserved on submission) as a "NewVersionOf" relation.